

Topaz: Topology-Aware Cache Placement for Asymmetric Memory Systems

Anonymous
Anonymous

Abstract

Looking at the direction of RAM, it's getting more and more asymmetric. NUMA has been the technology for large-scale servers, but the recent CXL shows that this direction is solidifying. In production caches, RAM is treated, allocated and used as if it is uniform. Topaz was built to answer the question: *does topology-aware placement for asymmetric memory systems in caches have any benefit?* Our results show that it does: 25x lower p50 GET latency (71ns vs 1804ns), 10x higher throughput, and 100% vs 50% memory efficiency compared to CacheLib [1] under YCSB [3] workloads, with flat scaling from 1 to 4 NUMA nodes. This was validated on AMD EPYC Milan (4 NUMA nodes) and Intel Xeon Ice Lake (2 NUMA nodes).

CCS Concepts: • Software and its engineering → Main memory.

Keywords: NUMA, cache, topology-aware placement, ARC, memory hierarchy

ACM Reference Format:

Anonymous. 2026. Topaz: Topology-Aware Cache Placement for Asymmetric Memory Systems. In *Proceedings of 2026 USENIX Annual Technical Conference (ATC'26)*. ACM, New York, NY, USA, 6 pages.

1 Introduction

The future of RAM is heading towards fragmented architecture. This change brings challenges. CXL, for example, suffers from lack of software support as argued by [10] and the point still stands for NUMA: despite the age of the technology, a general support or abstraction doesn't exist in userspace. The hardware expansion is heading into a direction but the software is not following.

Our focus was on cache systems. Modern servers expose non-uniform memory access: on AMD EPYC Milan with 4 NUMA nodes, cross-socket access has a distance ratio of 3.2x over local (distance 32 vs. 10). Under the default OS interleaving policy, every store is potentially interleaved across nodes, exposing further latencies. Without topology awareness, every cache access is a potential cross-socket round-trip. Available solutions largely ignore this asymmetry. CacheLib has no NUMA binding¹ and 50% memory overhead from slab allocator [2]. RocksDB block cache [4] has sharded LRU

¹This claim stands only for the open-source version. We do not know what Meta's internal production version implements.

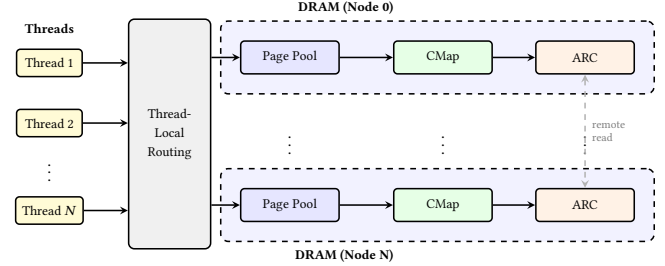


Figure 1. Topaz architecture. Each thread is routed to its local NUMA node via thread-local routing. Each node owns an independent page pool, hash map (CMap), and ARC eviction policy. Cross-node reads are best-effort and performed directly at the ARC layer; writes are always local.

but no topology awareness. memcached [6] is also not topology-aware and is single-threaded per core. In all stated examples, memory is treated as uniform.

Topaz bridges this gap by making memory topology a first-class input to placement decisions. The design is built around:

- Per-node page pools: one physical allocation per NUMA node, bump allocator, 100% memory efficiency contrasted with CacheLib's slab allocator [2] with 50% efficiency.
- Thread-local key routing: read-your-own-writes guaranteed, cross-node is best-effort (standard cache semantics)
- Per-node lock partitioning + ARC eviction: zero cross-node contention on hot path
- Cross-platform evaluation on Intel (2-node) and AMD (4-node) under YCSB [3]: up to 25x lower latency, 10x higher throughput

Cautionary finding: alignas(64) stats caused 6.6x cross-socket regression, fixed by per-node stat sharding.

2 Architecture

Topaz is organized as N independent per-node mini-caches, one per NUMA node. Each mini-cache owns its data exclusively: a physical page pool allocated on-node via `numa_alloc_onnode`, a concurrent hash map (CMap), and an ARC eviction policy instance. Threads are assigned to the NUMA node they run on; keys written by a thread are stored on that thread's node. This eliminates cross-node synchronization on the common case of read-after-write.

2.1 Per-Node Page Pools

At instance creation and for each node, Topaz allocates using `numa_alloc_onnode` a physical arena, which it carves into logical fixed-sized pages (4 KB). The physical memory is owned by a single node. The pages act as a store where values are stored via a bump allocator. Keys are also stored inline (`atomic<size_t>` for next-free). This guarantees all data for a node resides on that node's DRAM and importantly, per-node data separation. The architecture also achieves 100% memory efficiency, no slab overhead as opposed to CacheLib's slab allocator [2] with 50% overhead from meta-data + size-class fragmentation.

2.2 Thread-Local Key Routing

A key idea was using the requesting thread's current NUMA node as a home node. The node's routing state is stored in `thread_local` to avoid syscall per access and enforce thread-locality. In each node, Get first checks its local store for the key; if it's not found, it scans other nodes. While that on its own doesn't avoid cross-node reads, Topaz's architecture allows for some very interesting consequences through these combined guarantees:

- Read-after-write workloads (common case): local check always hits.
- Read-your-own-writes guaranteed within node.
- Keys are not duplicated across nodes.

That combination allows Topaz to simply side-step the need to synchronize between nodes for Get. The reason is that the uniqueness of keys ensures that there is only one writer for any key, so protecting the return slot is not needed².

A round-robin approach was considered for key placement, but even distribution goes against locality. It also loses the shortcut of local-first fetching.

2.3 Per-Node CMap and ARC

Each node operates its own mini-cache, composed of a CMap index and an ARC [12] eviction policy instance³. The caches are per-node independent⁴, but coupled through Topaz's routing layer. The node's reads and writes are protected and organized with a seqlock [8] for lock-free reads and CAS for writes. ARC uses a lock-free promotion buffer (MPSC) for deferred promotions to avoid congesting the seqlocks.

CMap is a Swiss table [9] variant designed with concurrency in mind, adding SIMD probing alongside per-slot seqlock protection⁵. This makes reads and writes purely lock-free operations. The design allows ARC's implementation to be just a policy without adding complexity to it.

²This allows for Topaz to use broadcast races for multi-node fetches without any result slot sync. This is a future direction.

³Topaz uses an exact implementation of ARC.

⁴Inter-node independence.

⁵Not related to the mini-cache's seqlock.

Each node also has a per-node job queue; each node spawns threads pinned to the NUMA node as background workers alongside a dedicated maintenance thread.

2.4 Per-Node Statistics

Along the development and benchmarking for the library, we went from running the tests on an Intel Xeon Ice Lake machine (2 NUMA nodes meaning two mini-caches and two operating threads) to AMD EPYC Milan (4 NUMA nodes, double the caches and threads). Prior thread-scaling benchmarks (on Xeon Ice Lake) showed that the system should scale linearly but when we ran the benchmark harness the numbers were off. At 2 threads p50 GET was 120ns but at 4 threads p50 GET was 587ns. This looked anomalous and so we investigated the issues. At first we assumed that a contention on the spinlock was the culprit, then we looked into the maintenance thread's drain logic and it wasn't causing the bump in latency. Worse yet, the latency issue only showed up when we implemented the maintenance thread, and that leads to assume that it was the cause. To our dismay, any attempt to fix the maintenance thread or other components, either made the system worse or did nothing. So we resorted to a methodological audit of all the data structures of Topaz, assessing what is shared, what is under contention. The surprise (and irony) was that the only shared structure between the entire Topaz architecture was the Statistics class which had `14 alignas(64) std::atomic<> fields in heap-allocated cache object (on node 0)`.

On a single socket `alignas(64)` prevents false sharing between counters but across sockets it MAXIMIZES cross-socket transfer (which was catastrophic).

What it caused was:

- 3 exclusive cache-line migrations per GET at ~120ns each on Infinity Fabric.
- Added ~360ns per GET.

To fix it, we moved hot-path counters to per-node atomics in NUMA-allocated `PerNodeDetails`⁶. We also made sure each thread writes to its own on-node counters and a background aggregation drains to global counters every 2ms.

The results were:

- 4T p50 GET: 587ns → 71ns (6.6x improvement)
- 2T p50 GET: 120ns → 74ns (39% improvement)
- Now matches single-thread baseline at 70ns.

Our cautionary lesson about `alignas(64)` is:

- It is not a substitute for NUMA-aware placement of performance counters
- Correct for one topology tier (intra-socket false sharing) — harmful for another (inter-socket bouncing)

⁶The struct containing all per-node state, including CMap and ARC.

2.5 Staging Pages

The initial benchmarks showed terrible SET latency (p99 at 68,000–280,000 ns). The root cause was that ARC evicts keys efficiently, but the bump allocator must scan all pages $O(P)$ to find a reclaimed slot. We implemented staging pages: the last page(s) in each node’s pool serve as an $O(1)$ overflow target for SET, decoupling the write path from space management. A background maintenance thread relocates keys from the staging page into sparse pages (page drain compaction), creating a virtuous cycle: eviction empties pages, drain consolidates, and recycled pages become fresh bump targets. This reduced p99 SET 24–63 $\times$ (to 2,800–4,500 ns) with zero GET regression.

3 Evaluation

3.1 Methodology

For the benchmarks, we ran them on two hardware architectures:

- c6a.metal: AMD EPYC 7R13 Milan, 4 NUMA nodes, 192 vCPUs, 32MB L3 per CCD, Nodes (0,1)=socket 0, (2,3)=socket 1.
- c6i.metal: Intel Xeon Platinum 8375C Ice Lake, 2 NUMA nodes, 128 vCPUs, 56MB L3 per socket.

Both machines had turbo disabled.

The systems compared are:

- Topaz-TL (thread-local routing) — our system.
- Topaz-SN (single-node, all threads share one node) — ablation.
- CacheLib [1] (Facebook production cache, LRU).
- RocksDB [4] (standalone NewLRUCache).
- TBB [13] (unbounded concurrent_hash_map, throughput ceiling).

The benchmarks featured two workload suites:

- YCSB [3] standard A (50R/50W), B (95R/5W), C (100R) at capacities 64–512 MB.
- A custom partitioned-key harness (Zipfian $\theta=0.99$) for thread-scaling analysis at 32 MB.

Each YCSB workload ran for 10 iterations (independent fresh-system runs) with 200K ops per thread per iteration across value sizes 64 B, 256 B, and 1024 B. Thread-scaling benchmarks use 64 B values with partitioned Zipfian access per thread. We captured the mean $\pm$ std across iterations.

System configurations. Topaz-TL splits total capacity equally across N NUMA nodes, allocates per-node via `numa_alloc_local`, and pins worker threads to their home node. Topaz-SN places all memory on node 0 with no pinning, isolating the effect of topology-aware placement. CacheLib uses its default LRU allocator with a single pool; its slab allocator requires approximately 2 $\times$ the target capacity for internal fragmentation, so we provision accordingly. RocksDB uses NewLRUCache with default sharding and `high_pri_pool_ratio=0`. TBB uses `concurrent_hash_map` with no capacity limit, serving

as a concurrency baseline rather than a capacity comparison. All systems operate on the same key space and value sizes. Threads for Topaz-TL are pinned to NUMA nodes; all other systems use OS-default scheduling.

3.2 Capacity Sweep

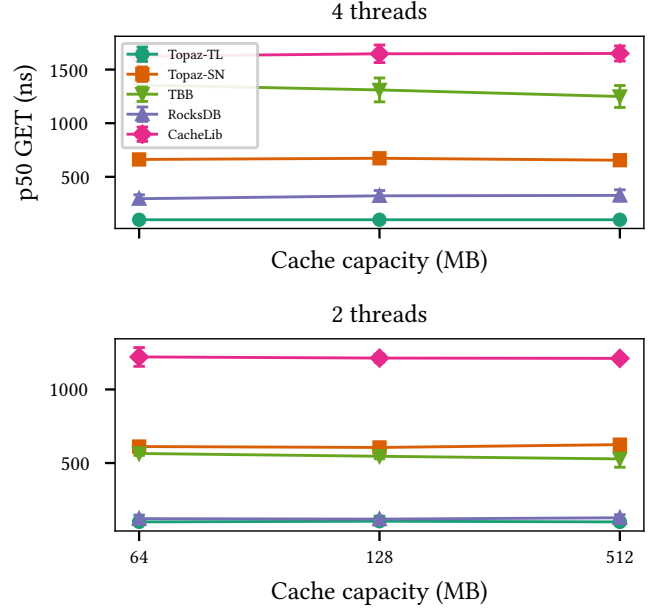


Figure 2. p50 GET latency vs. cache capacity for YCSB-B (64B values). Left: 4 threads. Right: 2 threads. TopazTL remains flat at 100 ns across all capacities. NUMA-blind systems pay a fixed 500–1600 ns penalty that does not shrink with more memory. Error bars show ± 1 std over 10 iterations.

3.3 Workload Sensitivity

3.4 TopazTL vs CacheLib Head-to-Head

3.5 Value Size and Hit Rate

3.6 Thread Scaling

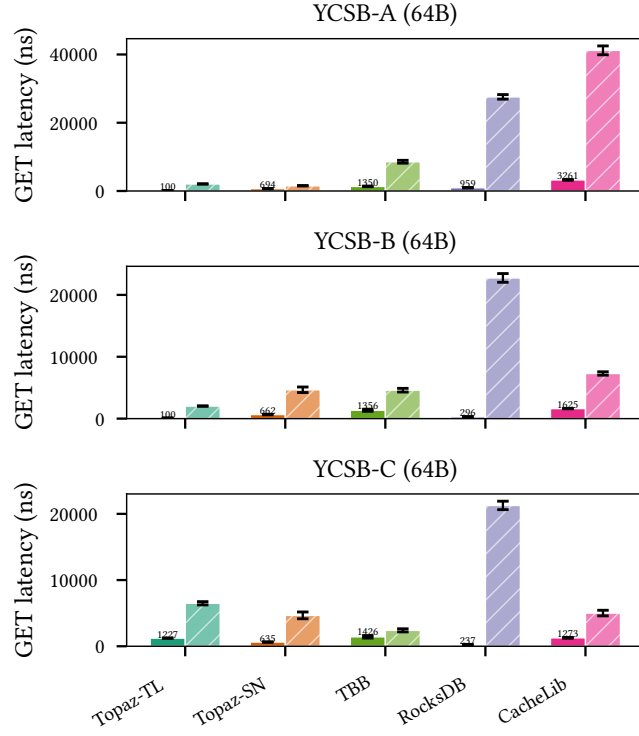


Figure 3. GET latency for YCSB workloads A, B, C (64B values, 4 threads, 64 MB cache). Solid bars: p50. Hatched bars: p99. Error bars show ± 1 std over 10 iterations.

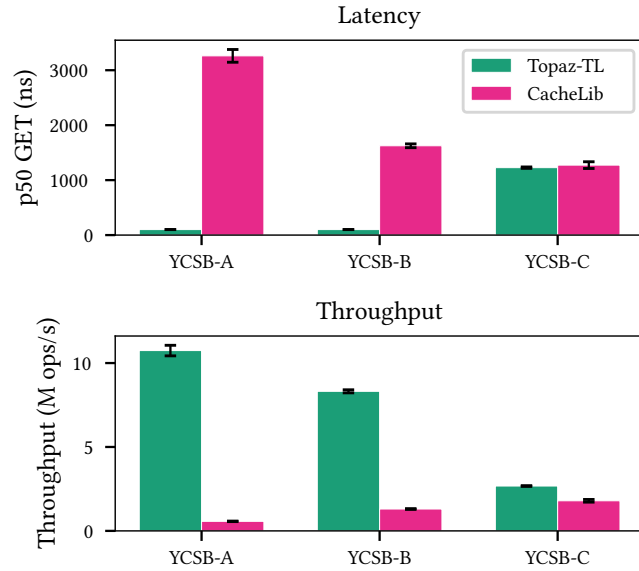


Figure 4. TopazTL vs. CacheLib head-to-head on YCSB A/B/C (64B, 4 threads, 64 MB cache). Top: p50 GET latency. Bottom: throughput. Error bars show ± 1 std over 10 iterations.

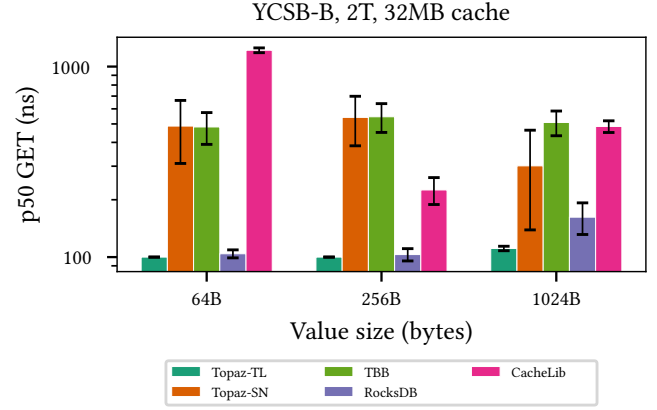


Figure 5. YCSB-B p50 GET latency across value sizes (2 threads, 32 MB cache). Error bars show ± 1 std over 10 iterations. TBB is unbounded (100% hit rate at all sizes).

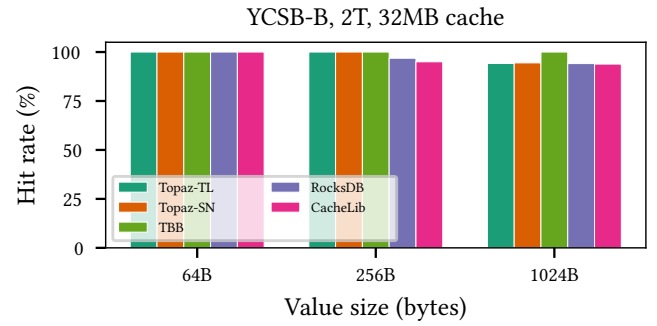


Figure 6. Hit rate for YCSB-B across value sizes (2 threads, 32 MB cache). TBB is unbounded (100% at all sizes, not shown).

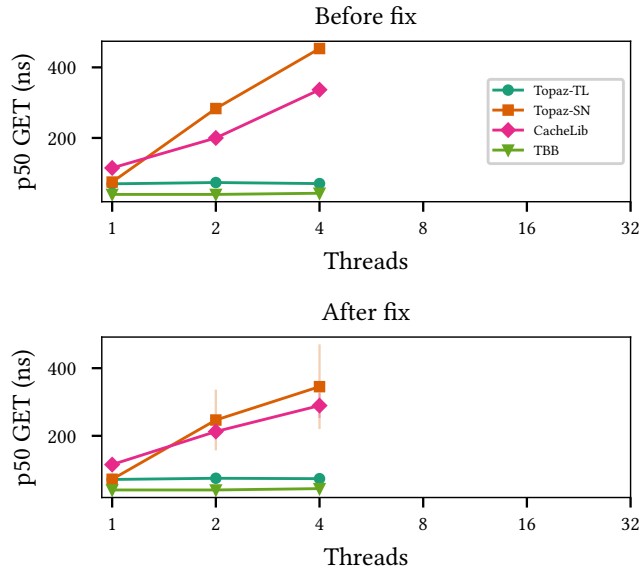


Figure 7. Thread scaling on AMD EPYC Milan (4 NUMA nodes, 64 MB pool). Top: before per-node stat fix. Bottom: after fix. Error bars show ± 1 std over 10 iterations. Before: TopazTL degrades $6.6\times$ at 4T. After: flat at 70 ns.

4 Discussion

About the locality assumption:

Thread-local routing assumes read-after-write locality, this holds for web caches, database buffer pools and CDN edge caches. but there is an adversarial case: all the writes occur in node 0, reads on node 1 — every TL read becomes cross-node.

An idea is a migration policy, where keys read more cross-node gets moved to the storage of the nodes that needs it most, this needs to be paired with hysteresis to avoid bouncing.

5 Related Work

NUMA-aware allocation:

numactl, libnuma are node-binding primitives, OS-specific and they leave policy to the application. Unlike the prior two, Windows NUMA APIs is, frankly, bare-bones. It offers allocation and pinning but not the same level of control nor flexibility of Unix libraries. You can't even poll for topology distance. jemalloc [5], tcmalloc [7] are NUMA-aware but general-purpose allocators, not caches. CoDB [11] is nearly the same, it is a NUMA-friendly malloc but — again — an allocator not cache.

6 Conclusion

References

- [1] Benjamin Berg, Daniel S. Berger, Sara McAllister, Isaac Grosof, Sathya Gunasekar, Jimmy Lu, Michael Uhlar, Jim Carrig, Nathan Beckmann, Mor Harchol-Balter, and Gregory R. Ganger. 2020. The CacheLib Caching Engine: Design and Experiences at Scale. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, Berkeley, CA, 753–768.
- [2] Jeff Bonwick. 1994. The Slab Allocator: An Object-Caching Kernel Memory Allocator. In *Proceedings of the USENIX Summer Technical Conference*. USENIX Association, Berkeley, CA.
- [3] Brian F. Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. 2010. Benchmarking Cloud Serving Systems with YCSB. In *Proceedings of the 1st ACM Symposium on Cloud Computing (SoCC)*. ACM, New York, NY, 143–154.
- [4] Siying Dong, Matthew Callaghan, Leonidas Galanis, Dhruba Borthakur, Tony Savor, and Michael Strum. 2017. Optimizing Space Amplification in RocksDB. In *Proceedings of the 8th Biennial Conference on Innovative Data Systems Research (CIDR)*. www.cidrdb.org, 1–1.
- [5] Jason Evans. 2006. A Scalable Concurrent malloc(3) Implementation for FreeBSD. In *Proceedings of the BSDCan Conference*. Ottawa, Canada.
- [6] Brad Fitzpatrick. 2004. Distributed Caching with Memcached. *Linux Journal* 2004, 124 (2004), 5.
- [7] Sanjay Ghemawat and Paul Menage. 2009. TCMalloc: Thread-Caching Malloc. <https://goog-perftools.sourceforge.net/doc/tcmalloc.html>.
- [8] Thomas E. Hart, Paul E. McKenney, and Angela Demke Brown. 2006. Making Lockless Searches Fast. *ACM SIGPLAN Notices* 41, 11 (2006), 56–63.
- [9] Stefan Kröten, Djamal Belazzougui, and Annette Bieniusa. 2023. Engineering High-Performance Hash Tables for Modern Hardware. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. ACM, New York, NY, 1–14.
- [10] Philip Levis, Kun Lin, and Amy Tai. 2023. A Case Against CXL Memory Pooling. In *Proceedings of the 22nd ACM Workshop on Hot Topics in Networks (HotNets '23)*. ACM, New York, NY, USA. doi:10.1145/3626111.3628195
- [11] Tongping Liu, Charlie Curtsinger, and Emery D. Berger. 2018. CoDB: A Concurrent, NUMA-Friendly malloc. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. ACM, New York, NY, 1–14.
- [12] Nimrod Megiddo and Dharmendra S. Modha. 2003. ARC: A Self-Tuning, Low Overhead Replacement Cache. In *Proceedings of the 2nd USENIX Conference on File and Storage Technologies (FAST)*. USENIX Association, Berkeley, CA, 115–130.
- [13] James Reinders. 2007. Intel Threading Building Blocks: Outfitting C++ for Multi-core Processor Parallelism. O'Reilly Media, Sebastopol, CA.